

SOFTWARE QUALITY & TESTING

UNIT - I: Software Quality Assurance:

Software Quality: Definition, Importance in SDLC, Cost of Quality (COQ): Prevention, Appraisal, and Failure Costs, Quality vs Reliability vs Security vs Maintainability.

Software Quality Models: McCall's Quality Factors, Boehm's Quality Model, CMMI, SQA Team Roles and Skills, SQA vs QC vs Testing

Q1. Define Software Quality. Why is it important in SDLC?

Definition:

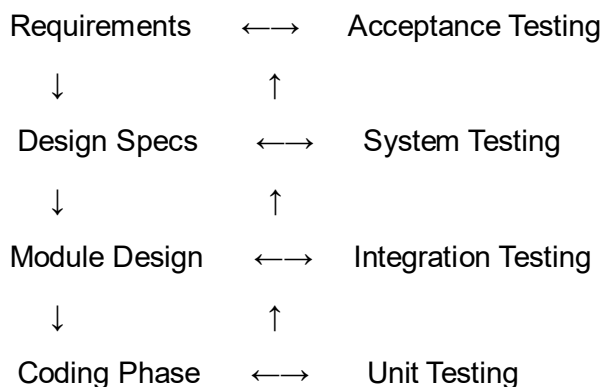
Software quality is the degree to which software meets specified requirements, customer expectations, and is free from defects.

Importance in SDLC:

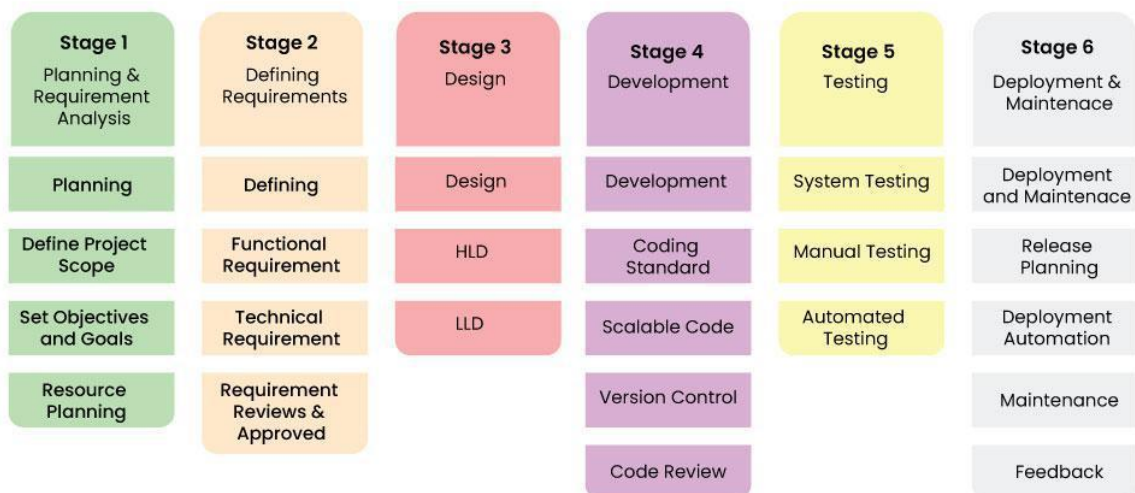
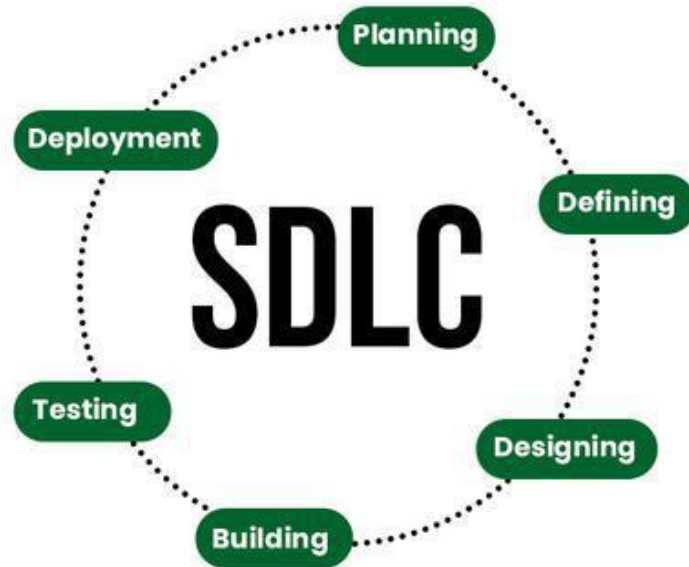
- Ensures requirement completeness and correctness.
- Reduces cost of rework.
- Enhances customer satisfaction and trust.
- Increases maintainability and reusability.
- Reduces risks of failures post-deployment.
-

✎ **Diagram:** V-shaped SDLC model highlighting quality checkpoints

Structure:



Label: "Quality checks at each corresponding stage ensure early detection and prevention of defects."



6 Stages of Software Development Life Cycle



Reference: Daniel Galin, *Software Quality Assurance—From Theory to Implementation*

Q2. Explain Cost of Quality (COQ) and its components.

Definition:

COQ is the total cost involved in achieving and maintaining software quality.

Components:

Cost Type	Description	Examples
Prevention Cost	Cost to prevent defects	Training, process audits

Cost Type	Description	Examples
Appraisal Cost	Cost to detect defects	Testing, inspections
Internal Failure Cost	Defects before delivery	Rework, bug fixes
External Failure Cost	Defects after delivery	Customer support, recalls

✂ **Diagram:** Pie chart or bar chart showing COQ distribution

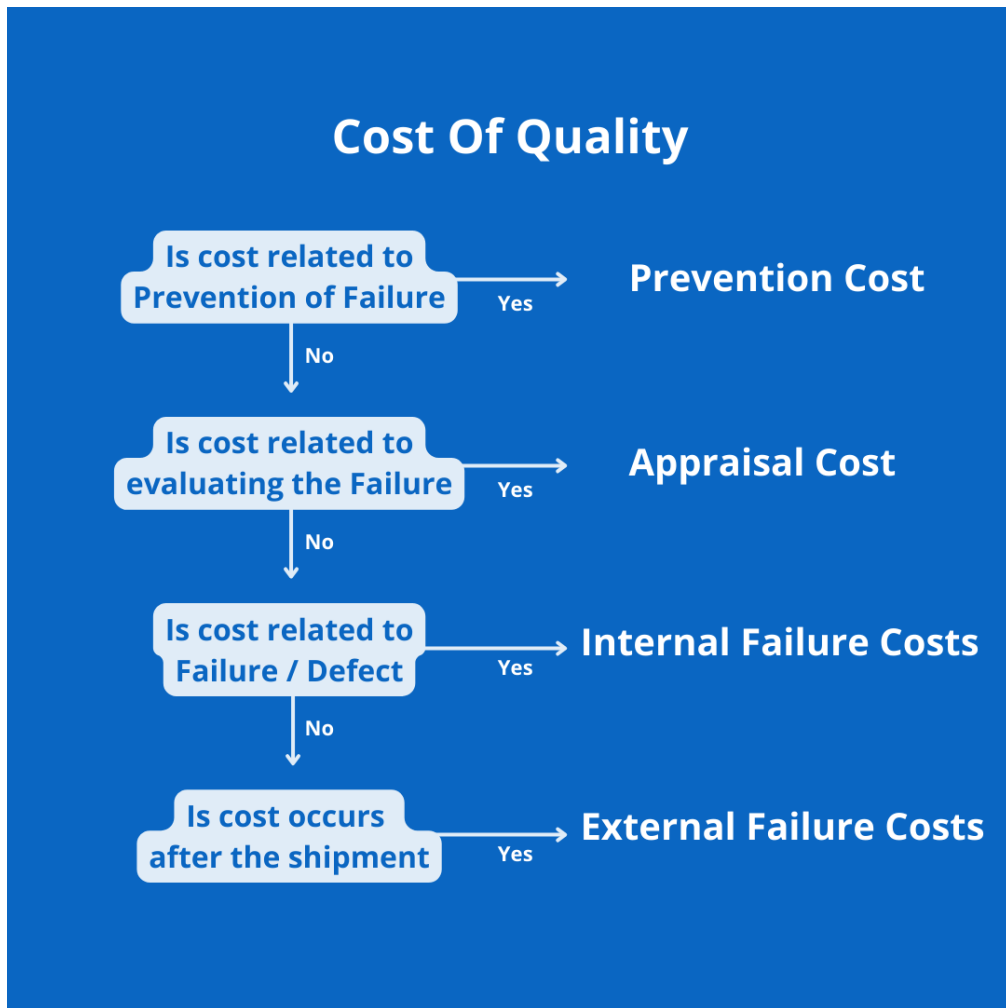
Structure (Pie Sections):

- □ Prevention Costs – 25%
- □ Appraisal Costs – 30%
- □ Internal Failure – 20%
- □ External Failure – 25%

✂ Bar Chart Alternative:

Cost Type	Relative Value
Prevention	██████████
Appraisal	████████████████████
Internal Failure	██████████
External Failure	████████████████████

Label: "Prevention + Appraisal = Quality investment; Failure = Quality loss"



Reference: Jeff Tian, *Software Quality Engineering*

Q3. Compare: Quality vs Reliability vs Security vs Maintainability

Attribute	Definition	Focus Area	Example
Quality	Fitness for use, defect-free	Holistic performance	Adherence to client specs
Reliability	Ability to operate without failure	Consistency, uptime	Server uptime > 99%
Security	Protect data against threats	Confidentiality, integrity	Secure login, encryption
Maintainability	Ease of making changes/fixes	Code flexibility	Modular, well-commented code

Diagram: 4-column Comparison Table

Attribute	Focus Area	Metric Used	Example
-----------	------------	-------------	---------

Attribute	Focus Area	Metric Used	Example
Quality	Overall correctness	Compliance rate	Meets all client specs
Reliability	Error-free duration	MTBF (Mean Time B/W Failures)	Server uptime 99.99%
Security	Data protection	Vulnerability Score	No unauthorized access
Maintainability	Ease of change	Change Lead Time	Bug fix in <24 hrs

What is Reliability, Availability, and Maintainability



Reliability

The term Reliability represents the probability that an asset will operate without failure in a given time under strict conditions.



Maintainability

The term maintainability describes how easy and resource intensive it is to perform maintenance on an asset.



Availability

Availability provides the probability that an asset is in operable condition at a given time.

The purpose of RAM analysis is to identify major causes of loss of operational availability or other issues that may limit the production throughput. The end goal is to improve asset design and maintenance programs, optimizing the overall life cycle (LCC) costs of your assets.



 **Reference:** Sagar Naik, *Software Testing and Quality Assurance*

Q4. Describe McCall's Quality Model with its categories.

McCall's Model (1977):

Divided into **3 major categories**:

1. **Product Operation:** Correctness, Efficiency, Integrity, Usability
2. **Product Revision:** Maintainability, Flexibility, Testability
3. **Product Transition:** Portability, Reusability, Interoperability



Diagram: 3-layered triangle model of McCall's quality factors

Structure:

[Product Transition]

- Portability
- Reusability
- Interoperability

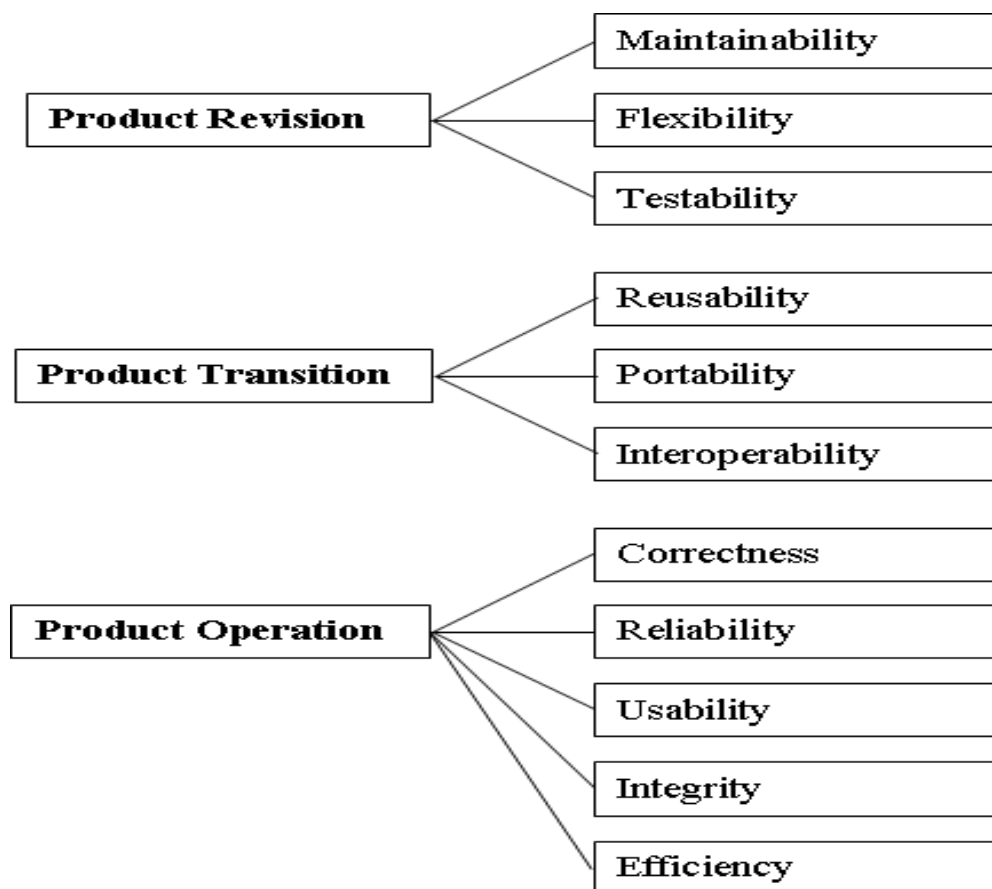
[Product Revision]

- Maintainability
- Flexibility
- Testability

[Product Operation]

- Correctness
- Reliability
- Efficiency
- Integrity
- Usability

🔑 **Pyramid Tip:** Operation at the base → Transition at top





Reference: William Perry, *Effective Methods for Software Testing*

Q5. Describe Boehm's Quality Model. Compare it with McCall's Model.

Boehm's Model (1978):

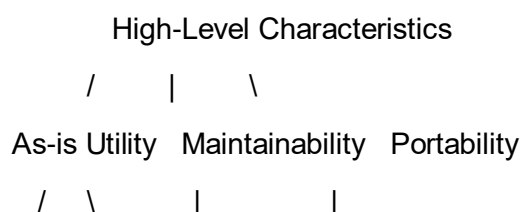
- Hierarchical model with **High-Level, Intermediate, and Primitive** levels.
- Example attributes: As-is Utility, Portability, Human Engineering.

Comparison:

Feature	McCall	Boehm
Structure	Category-based	Hierarchical
Focus	Product attributes	User perspective
Year	1977	1978

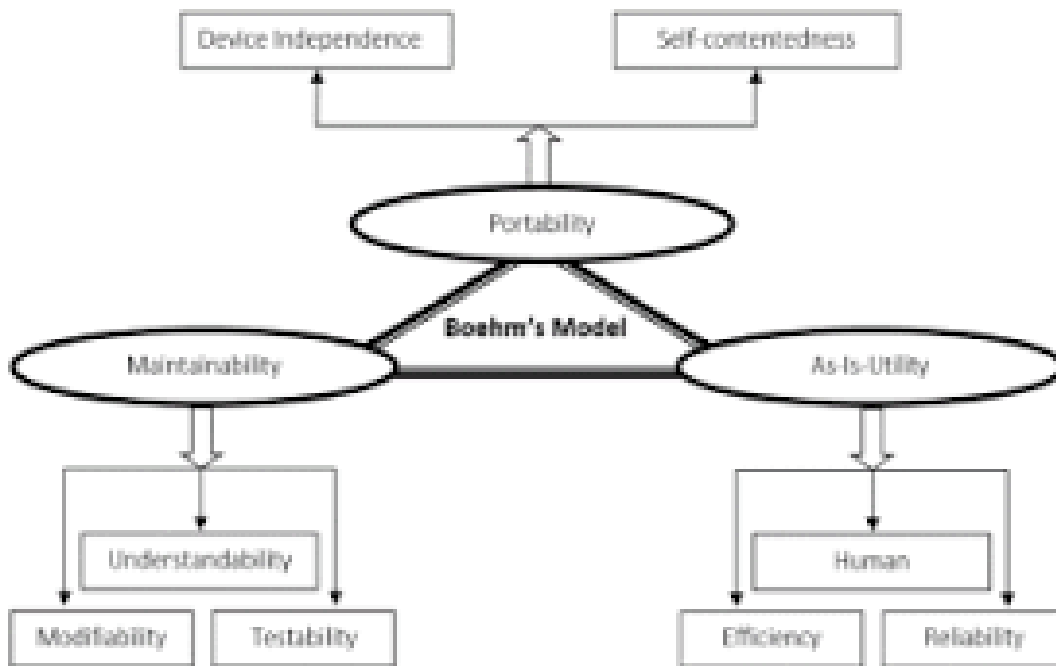
Diagram: Tree structure showing Boehm's hierarchy

Structure:



Reliability Efficiency Testability Device Independence

Label: Top-down decomposition from general to specific quality factors.



Boehm's Quality Model



Software Quality Assurance (SQA)

Q6. Explain CMMI and its levels.

CMMI (Capability Maturity Model Integration):

A process improvement model used to assess and improve software processes.

Levels:

1. **Initial** – Ad hoc, chaotic
2. **Managed** – Project-level process control
3. **Defined** – Organization-level process standardization
4. **Quantitatively Managed** – Measurable quality control
5. **Optimizing** – Continuous improvement

 **Diagram:** CMMI maturity staircase model

Structure:

[5] Optimizing

[4] Quantitatively Managed

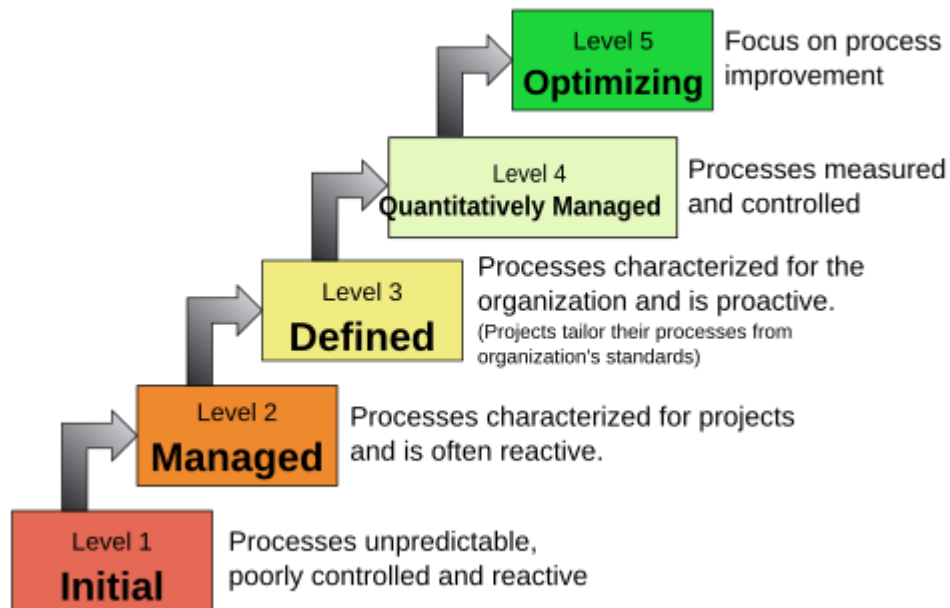
[3] Defined

[2] Managed

[1] Initial

Label: "Process maturity grows from chaos to optimization."

Characteristics of the Maturity levels

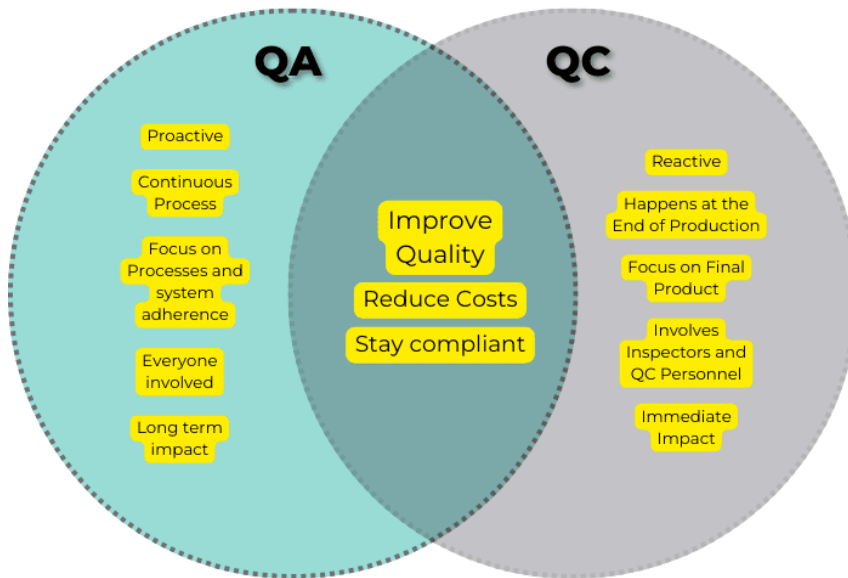


 **Reference:** Daniel Galin

Q7. Differentiate between SQA, Quality Control (QC), and Testing.

Aspect	SQA	QC	Testing
Orientation	Process	Product	Product
Type	Preventive	Detective	Detective
Timing	Entire SDLC	End of development	Before release
Methods	Reviews, audits	Inspections, peer reviews	Manual/automated tests

Quality Assurance vs. Quality Control



Q8. What are the roles and skills required for an SQA team?

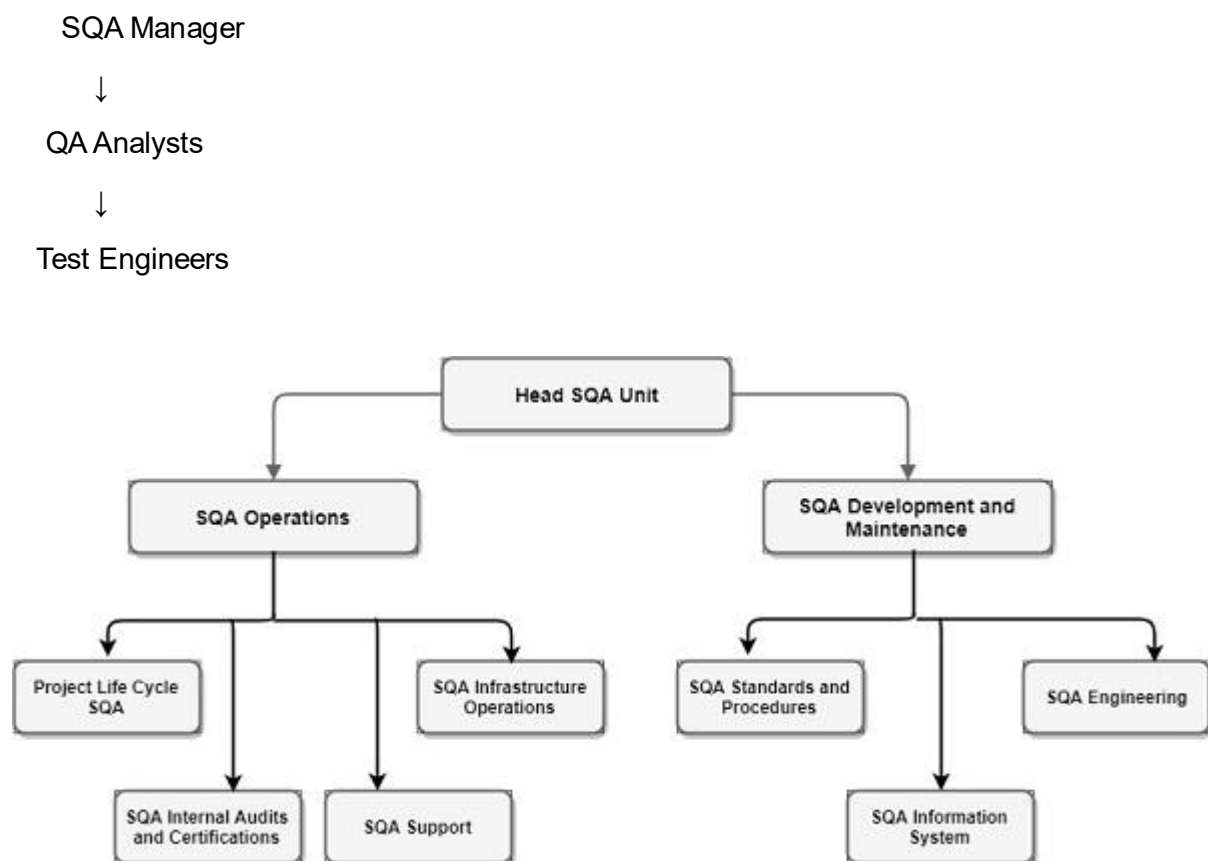
Roles:

- **SQA Manager** – Oversees QA strategy and planning.
- **QA Analyst** – Conducts reviews, audits.
- **Test Engineer** – Designs and executes test cases.

Skills:

- SDLC/STLC knowledge
- Tools (JIRA, Selenium, Git)
- Documentation & communication
- Basic scripting/automation

Diagram Type: Org Chart or Hierarchy Tree



Reference: Jeff Tian

Q9. How does SQA impact software maintenance and reliability?

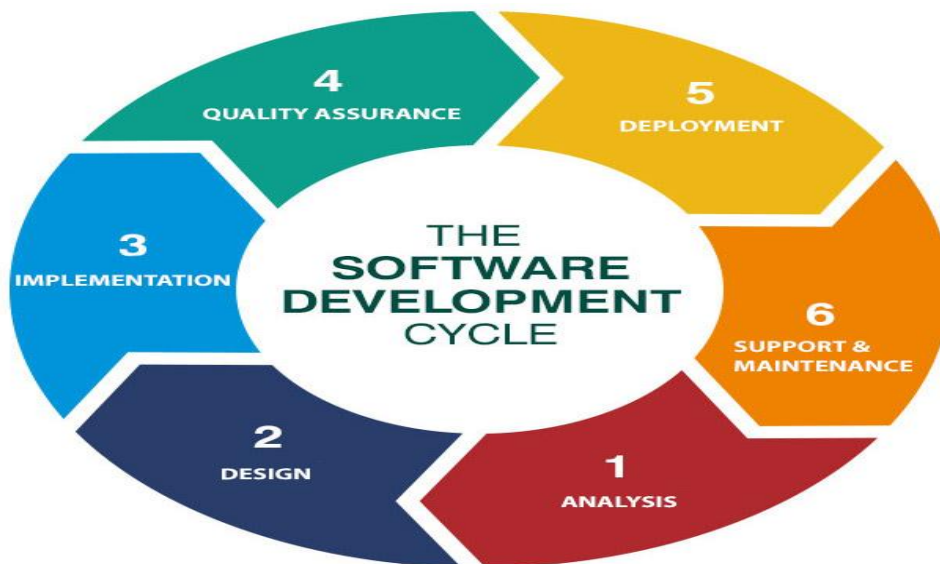
- Promotes standardized coding practices.
- Encourages documentation and traceability.
- Reduces effort and cost of bug fixes.

- Enhances system uptime by minimizing failure points.

✎ **Diagram:** Lifecycle loop showing SQA impact

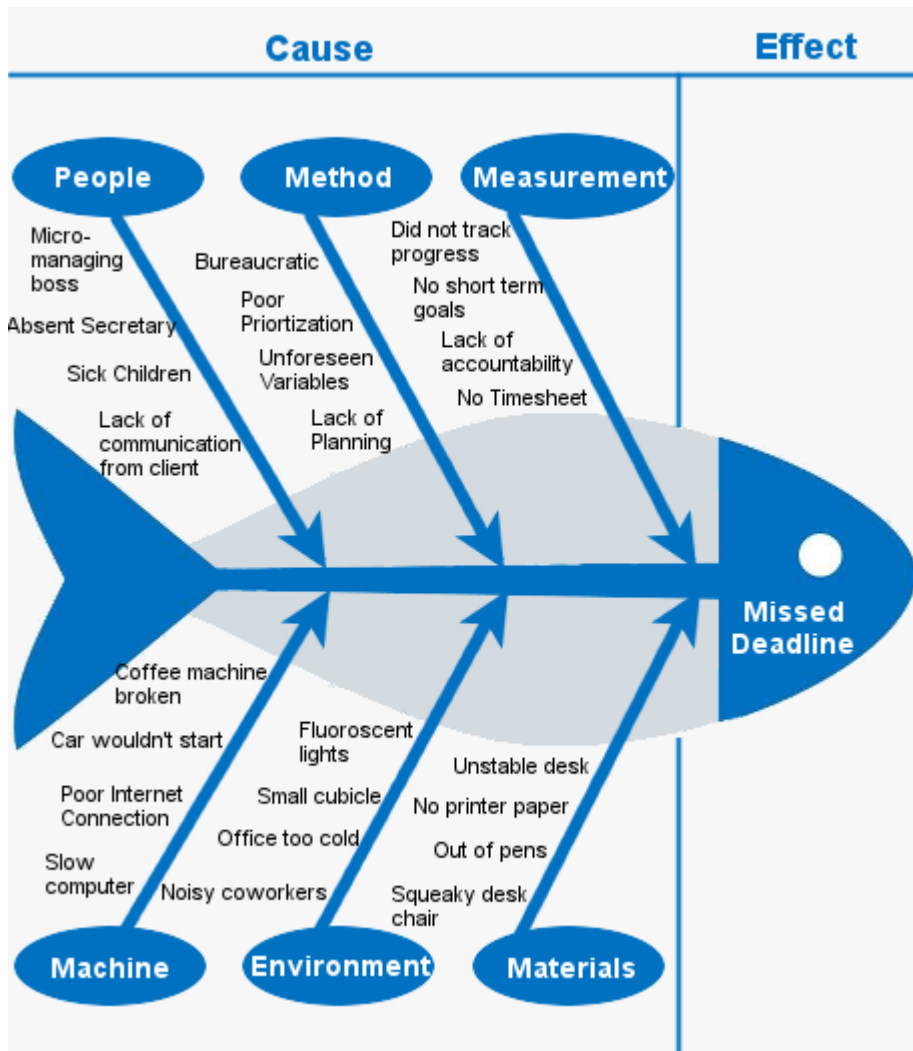
Stages:

- Development → Deployment → Feedback → Bug Fixes → Updates → Back to Deployment



Label: "SQA reduces breakdowns, simplifies maintenance, and supports long-term reliability."

📖 **Reference:** William Perry



Q10. Why is prevention better than detection in software quality?

- Cheaper to prevent bugs during design than to fix them after release.
- Increases productivity and team morale.
- Leads to cleaner, maintainable code.

Diagram Type: Line Graph

X-axis: SDLC Phases

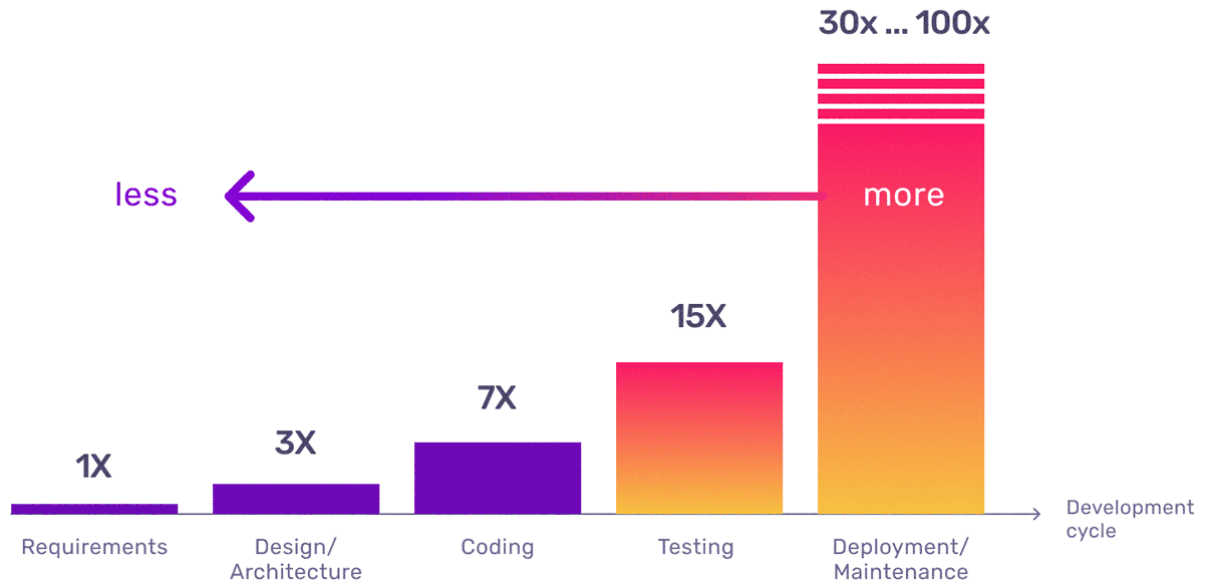
Y-axis: Cost of Defect Fix

Two lines:

- □ **Prevention Cost:** Flat or low
- ● **Detection Cost:** Exponentially rising

✂ **Label:** "Cost to fix a defect increases 10x with each phase delay."

Cost of Defects



The more time we save your team, the more time they have to find bugs sooner.

That Saves Money

 **Reference:** Daniel Galin

